

JAVA

List & ArrayList

ArrayList • LinkedList • Iterator • Autoboxing • enum

Apostila completa com explicações detalhadas, exemplos de código e mini-projects práticos.

CONTEÚDO DA APOSTILA

- 01 Interface List e coleções dinâmicas
- 02 ArrayList — funcionamento interno
- 03 Arrays vs. ArrayLists
- 04 Big O Notation e memória
- 05 LinkedList — Queue e Stack
- 06 Iterators e ListIterator
- 07 Autoboxing e Unboxing
- 08 enum type
- 09 Mini-Projects práticos

1

List e ArrayList

Interface List, ArrayList e comparação com Arrays

O que é uma List em Java?

Em Java, uma **List** não é uma classe concreta — é uma **interface**. Isso significa que ela define um conjunto de comportamentos (métodos) que qualquer classe que a implemente deve fornecer, sem ditar como esses comportamentos devem funcionar internamente. A interface List descreve uma coleção ordenada de elementos acessíveis por índice inteiro, que aceita duplicatas e permite elementos *null*.

A grande vantagem de trabalhar com a interface List (e não diretamente com ArrayList ou LinkedList) é o desacoplamento: você pode escrever código que opera sobre qualquer lista, e trocar a implementação concreta em um único ponto sem alterar o restante do programa. Essa prática é chamada de **coding to an interface** e é considerada uma das melhores práticas da orientação a objetos.

As principais implementações de List são: **ArrayList**, que usa um array interno redimensionável, e **LinkedList**, que usa nós ligados em sequência. Cada uma tem pontos fortes e fracos que veremos ao longo desta apostila.

```
List<String> groceries = new ArrayList<>();
groceries.add("Milk");
groceries.add("Eggs");
groceries.remove("Milk");
System.out.println(groceries.contains("Eggs")); // true
System.out.println(groceries.size());           // 1
```

Como o ArrayList funciona internamente?

O **ArrayList** armazena seus elementos em um array interno comum. Para evitar o custo de redimensionar constantemente, esse array é criado com uma **capacidade maior** do que o número atual de elementos. Por padrão, a capacidade inicial é 10. À medida que você adiciona elementos, o array vai sendo preenchido. Quando atinge o limite, o Java cria um novo array com capacidade maior e copia todos os elementos — um processo transparente para o programador.

A fórmula de crescimento padrão é: **nova capacidade = (capacidade antiga × 3/2) + 1**. Essa estratégia é chamada de crescimento amortizado e garante que, em média, cada inserção tenha custo constante $O(1)$, mesmo que eventualmente uma inserção precise disparar uma realocação $O(n)$.

O acesso por índice (get/set) é extremamente rápido — $O(1)$ — porque o Java calcula diretamente o endereço de memória com base no índice e no tamanho de cada elemento. Já a inserção ou remoção no meio da lista é $O(n)$, pois todos os elementos após o ponto de inserção precisam ser deslocados.

```
ArrayList<String> list = new ArrayList<>(); // capacidade inicial: 10
list.add("A"); // índice 0
list.add(0, "Z"); // insere no índice 0 – desloca todos
list.get(1); // "A" –  $O(1)$ 
list.set(0, "X"); // substitui "Z" por "X"
list.remove(0); // remove "X" – desloca elementos
```

Arrays vs. ArrayLists

Embora arrays e ArrayLists pareçam semelhantes à primeira vista — ambos são ordenados por índice e aceitam duplicatas — existem diferenças fundamentais entre eles. A mais importante: **arrays suportam tipos primitivos** (int, double, boolean...), enquanto o ArrayList só trabalha com **tipos de referência** (Integer, Double, Boolean...). Na prática, o Java faz a conversão automaticamente via autoboxing, mas é importante entender essa limitação.

Outra diferença crucial é o tamanho: arrays têm tamanho **fixo**, definido na criação — você não pode adicionar um elemento além do limite sem criar um novo array. O ArrayList, por outro lado, é **redimensionável automaticamente**. Além disso, o ArrayList implementa a interface List e oferece métodos convenientes como add(), remove(), contains() e sort(), que arrays simplesmente não têm.

■ Quando usar cada um?

Use Array quando souber o tamanho exato e precisar de primitivos ou desempenho máximo. Use ArrayList para coleções dinâmicas onde tamanho, conveniência e integração com Collections importam.

```
// Array – tamanho fixo, suporta primitivos
int[] arr = new int[5];

// ArrayList – redimensionável, só referências
ArrayList<Integer> list = new ArrayList<>();
list.add(1); list.add(2); list.add(3);
Collections.sort(list); // ordenar facilmente
```

2

Big O Notation & LinkedList

Memória, complexidade e a lista duplamente ligada

Arrays, ArrayList & LinkedList — Memória e Big O

Para escolher entre ArrayList e LinkedList, é fundamental entender como cada estrutura usa a memória e qual é o custo das operações mais comuns. A notação **Big O** descreve o crescimento do tempo de execução em função do tamanho da entrada, no pior caso.

O **ArrayList** (e o array simples) armazena seus elementos em blocos **contíguos de memória**. Isso significa que o processador pode calcular diretamente o endereço de qualquer elemento: $\text{endereço} = \text{base} + \text{índice} \times \text{tamanho_do_elemento}$. Por isso, `get(índice)` é $O(1)$ — constante, independente do tamanho da lista.

A **LinkedList** funciona de modo completamente diferente: cada elemento é armazenado em um **nó** que contém o valor, uma referência para o nó anterior e uma para o próximo. Os nós podem estar espalhados em qualquer lugar da memória — não há contiguidade. Para acessar o elemento de índice N , a LinkedList precisa percorrer os nós um a um, do início (ou do fim). Isso torna `get(índice)` $O(n)$.

Porém, a LinkedList se destaca na **inserção e remoção**: ajustar as referências dos nós vizinhos é $O(1)$, sem precisar deslocar nenhum elemento. O ArrayList, ao contrário, precisa deslocar todos os elementos após o ponto de inserção — $O(n)$. Portanto, a escolha ideal depende do padrão de uso da lista.

<code>get(índice)</code>	$O(1)$ ✓	$O(n)$
<code>add(no fim)</code>	$O(1)$ amortizado	$O(1)$ ✓
<code>add(no início/meio)</code>	$O(n)$	$O(1)$ ✓
<code>remove(índice)</code>	$O(n)$	$O(1)$ ✓
<code>contains()</code>	$O(n)$	$O(n)$

LinkedList — Queue, Stack e métodos exclusivos

A **LinkedList** em Java é única porque implementa não apenas a interface **List**, mas também **Queue** e **Deque** — tornando-se extremamente versátil. Ela pode ser usada como lista comum, fila, pilha ou fila dupla, dependendo dos métodos que você utilizar.

Como **Queue (fila — FIFO)**: o primeiro elemento inserido é o primeiro a ser removido. Use `offer()` para adicionar ao fim e `poll()` para remover do início. É a estrutura ideal para processar tarefas na ordem em que chegaram — filas de impressão, mensagens, etc.

Como **Stack (pilha — LIFO)**: o último elemento inserido é o primeiro a ser removido. Use `push()` para adicionar ao topo e `pop()` para remover do topo. É ideal para situações onde você precisa desfazer ações, avaliar expressões ou rastrear chamadas.

```
// LinkedList como Queue (FIFO):
LinkedList<String> queue = new LinkedList<>();
queue.offer("A"); queue.offer("B"); queue.offer("C");
queue.poll(); // remove "A" — chegou primeiro, sai primeiro

// LinkedList como Stack (LIFO):
LinkedList<String> stack = new LinkedList<>();
stack.push("X"); stack.push("Y");
stack.pop(); // remove "Y" — entrou por último, sai primeiro
```

3

Iterator & enum

Percorrer listas com segurança e constantes tipadas

Iterators — Como funcionam?

Um **Iterator** é um objeto que fornece uma forma padronizada de percorrer uma coleção elemento a elemento, sem expor a estrutura interna dessa coleção. Toda coleção Java que implementa a interface **Iterable** pode fornecer um Iterator através do método `iterator()`.

O Iterator tem um **cursor** que começa posicionado *antes* do primeiro elemento. A cada chamada de `next()`, o cursor avança uma posição e retorna o elemento atual. O método `hasNext()` verifica se ainda há elementos à frente — use-o sempre antes de chamar `next()` para evitar *NoSuchElementException*.

A grande vantagem do Iterator sobre o simples `for-each` é a possibilidade de **remover elementos durante a iteração** com `iterator.remove()`. Tentar remover elementos de uma lista com `for-each` enquanto a itera causa *ConcurrentModificationException* — um erro comum entre iniciantes. O Iterator resolve isso de forma segura.

O **ListIterator** (disponível apenas para `List`) é ainda mais poderoso: além de avançar com `next()`, ele permite retroceder com `previous()` — navegação bidirecional. É especialmente útil ao trabalhar com `LinkedList` para percorrer o itinerário de um caminho de ida e volta.

```
List<String> list = new ArrayList<>(List.of("A", "B", "C", "D"));
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    String s = it.next();
    if (s.equals("B")) it.remove(); // seguro — sem ConcurrentModificationException!
}
// list agora: [A, C, D]
```

O enum type — constantes tipadas e seguras

O tipo **enum** (enumeração) em Java é uma forma de declarar um conjunto de constantes predefinidas com nome e tipo — ao contrário de usar inteiros ou Strings soltas, que são propensos a erros. Um enum é na verdade uma **classe especial** que pode ter campos, construtores e métodos.

Cada constante de um enum possui duas propriedades automáticas: `ordinal()`, que retorna a posição (começando em 0), e `name()`, que retorna o texto exato da constante como String. Você também pode usar enums em expressões `switch`, tornando o código mais legível e seguro do que comparar inteiros ou Strings.

```
enum DayOfWeek { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY }

DayOfWeek day = DayOfWeek.FRIDAY;
System.out.println(day.ordinal()); // 4
System.out.println(day.name());    // "FRIDAY"

// enum em switch — mais seguro que String ou int:
switch (day) {
    case SATURDAY, SUNDAY -> System.out.println("Weekend!");
    default -> System.out.println("Dia de semana");
}
```

4

Autoboxing & Unboxing

Wrapper classes e conversão automática entre primitivos e objetos

Autoboxing e Unboxing — a ponte entre primitivos e objetos

O Java tem dois sistemas de tipos: os **tipos primitivos** (`int`, `double`, `boolean`, `char`...) e os **tipos de referência** (objetos). Coleções como `ArrayList` e `LinkedList` só conseguem armazenar objetos — nunca primitivos. Para contornar isso, o Java oferece as **wrapper classes**: `Integer`, `Double`, `Boolean`, `Character`, `Long`, `Short`, `Byte` e `Float`.

Boxing é a conversão de um primitivo para seu wrapper correspondente. **Unboxing** é o caminho inverso. Antes do Java 5, você precisava fazer essa conversão manualmente: `Integer.valueOf(42)` para boxing e `boxed.intValue()` para unboxing. Desde o Java 5, o compilador faz isso **automaticamente** — por isso se chama **autoboxing**.

O autoboxing é conveniente, mas traz um risco importante: se um wrapper tiver valor *null* e você tentar fazer unboxing, o Java lança um **NullPointerException** em tempo de execução — um erro silencioso e difícil de rastrear. Sempre verifique se o wrapper não é null antes de extrair seu valor primitivo.

```
// Boxing manual (antes do Java 5):
Integer boxed = Integer.valueOf(42);
int unboxed   = boxed.intValue();

// Autoboxing/Unboxing automático (Java 5+):
ArrayList<Integer> list = new ArrayList<>();
list.add(42);           // autoboxing:  int → Integer (automático)
int val = list.get(0);  // unboxing:   Integer → int (automático)

// Cuidado com null:
Integer x = null;
int y = x; // NullPointerException em tempo de execução!
```

■ Tabela de Wrappers

Cada primitivo tem seu wrapper: int→Integer, double→Double, boolean→Boolean, char→Character, long→Long, float→Float, short→Short, byte→Byte. Memorize essa correspondência!

5

Mini-Projects

Práticas guiadas — ArrayList, LinkedList e Autoboxing

Os mini-projects a seguir foram projetados para consolidar os conceitos aprendidos nesta apostila através de aplicações reais e progressivas. Cada mini-project apresenta um enunciado detalhado, as estruturas de dados recomendadas e dicas de implementação. Tente implementar cada um antes de consultar as dicas.

Mini-Project 1 — Grocery List App (ArrayList)

Construa um programa interativo de linha de comando para gerenciar uma lista de compras. O programa deve exibir um menu com as opções: **0 — Sair**, **1 — Adicionar item** e **2 — Remover item**. Após cada operação, a lista deve ser impressa em ordem alfabética.

Regras importantes: não permita itens duplicados (verifique com `contains()` antes de adicionar). Use `Scanner` para ler a escolha do menu e o nome do item, e trate opções inválidas com uma mensagem de erro amigável. A lista deve ser um `ArrayList<String>` e você deve usar `Collections.sort()` para ordenar.

```
ArrayList<String> groceries = new ArrayList<>();

// Adicionar sem duplicatas:
if (!groceries.contains(item)) {
    groceries.add(item);
    Collections.sort(groceries);
    System.out.println("Adicionado: " + item);
} else {
    System.out.println(item + " já está na lista!");
}
```

Mini-Project 2 — Travel Itinerary (LinkedList + ListIterator)

Crie um sistema de itinerário de viagem por cidades australianas, ordenadas por distância de Sydney. Para isso, implemente uma classe `Town` com os campos `name` e `distance`, e faça-a implementar `Comparable<Town>` para ordenar por distância.

Monte o itinerário em uma `LinkedList<Town>`. Implemente um menu interativo com as opções **F** (avançar para a próxima cidade) e **B** (voltar para a anterior), usando `ListIterator`. Atenção: ao mudar de direção (de B para F, ou de F para B), chame o método de movimento uma vez extra para compensar o comportamento do cursor do `ListIterator`.

```
LinkedList<Town> itinerary = new LinkedList<>();
// Adicionar cidades em ordem...

ListIterator<Town> it = itinerary.listIterator();
// Avançar:      Town t = it.next();
// Voltar:       Town t = it.previous();
// Mudar de B → F: it.next(); Town t = it.next(); // compensa o cursor
```

Mini-Project 3 — Banking App (Autoboxing + ArrayList)

Implemente um sistema bancário simples com duas classes principais. A classe `Customer` deve ter os campos `name` (`String`) e `transactions` (`ArrayList<Double>`), onde valores positivos representam créditos e valores negativos representam débitos.

A classe `Bank` deve ter o campo `name` e um `ArrayList<Customer>`. O banco deve suportar: adicionar um novo cliente (somente se não existir), adicionar uma transação a um cliente existente, e imprimir o extrato de um cliente, calculando o saldo por unboxing de cada `Double` da lista.


```
// Autoboxing ao adicionar transação:
customer.getTransactions().add(150.00); // double → Double (automático)
customer.getTransactions().add(-30.50); // débito

// Unboxing ao calcular saldo:
double balance = 0;
for (Double t : customer.getTransactions()) {
    balance += t; // Double → double (unboxing automático)
}
System.out.printf("Saldo: R$ %.2f%n", balance);
```

Resumo — O que você aprendeu nesta apostila

List	Interface Java que define coleções ordenadas por índice, com duplicatas e null permitidos.
ArrayList	Implementação de List com array interno redimensionável. get O(1), add/remove no meio O(n).
LinkedList	Lista duplamente ligada que também é Queue e Deque. get O(n), add/remove nas pontas O(1).
Big O	Notação para descrever a complexidade de algoritmos. ArrayList é melhor para acesso aleatório; LinkedList para inserções frequentes no início/fim.
Iterator	Objeto que percorre coleções com segurança, permitindo remoção durante a iteração.
enum	Tipo especial para constantes predefinidas com ordinal, name e suporte a switch.
Autoboxing	Conversão automática entre primitivos e wrappers (int↔Integer etc.) feita pelo compilador.